

Matplotlib Problems

Task 1: Basic Line Plot (Loss Curve)

Simulate and visualize model training convergence over epochs.

1. Create an array `epochs` from `1` to `10` .
2. Compute `loss = 1 / epochs` .
3. Create a plot showing `epochs` vs `loss` using a dashed red line with circle markers. Add grid lines and set axis labels (`Epochs` and `Loss`).

Task 2: Single Feature Scatter Plot

A researcher wants to visually inspect the relationship between patient age and total cholesterol.

1. Create a scatter plot using `plt.scatter` with `age` on the x-axis and `totChol` on the y-axis.
2. Set the points to blue with an alpha transparency of `0.5` .
3. Add a title: `"Age vs Total Cholesterol"` .

Task 3: Grouped Scatter Plot (Target Class Coloring)

Visualize feature separation between healthy and high-risk patients.

1. Create a scatter plot of `sysBP` (x-axis) vs `diaBP` (y-axis).
2. Color the points using the `TenYearCHD` target column (0 or 1) using the `'coolwarm'` colormap.
3. Label the x-axis `"Systolic Blood Pressure"` and y-axis `"Diastolic Blood Pressure"` .

Task 4: Feature Distribution Histogram

Examine the distribution of the `BMI` variable.

1. Plot a histogram of the `BMI` feature using `plt.hist` with 15 bins.
2. Set the bar color to `'skyblue'` with black edges (`edgecolor='black'`).
3. Label the x-axis `"BMI"` and y-axis `"Patient Count"` .

Task 5: Discrete Categorical Histogram

Check the distribution of daily cigarette consumption among patients.

1. Plot a histogram for `cigsPerDay` with 20 bins using `plt.hist` .
2. Set the color to `'green'` with an alpha transparency of `0.7` .
3. Add appropriate axis labels and a title.

Task 6: Customizing Plot Aesthetics & Legend

Create a line plot comparing two synthetic evaluation metrics over training steps.

1. Plot `train_acc` (e.g., `[0.6, 0.7, 0.8, 0.85, 0.9]`) and `val_acc` (e.g., `[0.55, 0.65, 0.75, 0.78, 0.8]`) against steps `[1, 2, 3, 4, 5]` .
2. Add custom labels to each line (`'Train Accuracy'` and `'Validation Accuracy'`).
3. Display a legend on the plot showing both label identifiers using `plt.legend()` .

Task 7: Side-by-Side Subplots Layout

Compare blood pressure metrics side-by-side using `plt.subplot` .

1. Create a figure layout with 1 row and 2 columns using `plt.subplot(1, 2, 1)` and `plt.subplot(1, 2, 2)` .
2. On the first panel, plot a histogram of `sysBP` .
3. On the second panel, plot a histogram of `diaBP` .
4. Apply `plt.tight_layout()` to prevent overlapping titles and labels.

Task 8: Stacked Subplot Layout

Examine heart rate and glucose distributions stacked vertically.

1. Create a 2-row, 1-column layout using `plt.subplot(2, 1, 1)` and `plt.subplot(2, 1, 2)`.
2. Plot a histogram of `heartRate` on the top subplot.
3. Plot a histogram of `glucose` on the bottom subplot.
4. Set distinct titles and labels for both subplots.

Task 9: Scatter Plot with Colorbar Mapping

Visualize how age relates to blood pressure alongside cholesterol levels.

1. Create a scatter plot of `age` vs `sysBP`.
2. Color the points based on `totChol` continuous values using `c=df['totChol']`.
3. Add a colorbar on the side using `plt.colorbar()` with the label `'Total Cholesterol (mg/dL)'`.

Task 10: Multi-Metric Loss & Accuracy Comparison

Create a 2-panel diagnostic figure for model training metrics.

1. Subplot 1 (`plt.subplot(1, 2, 1)`): Plot training loss (`[0.9, 0.5, 0.3, 0.1]`) and validation loss (`[0.95, 0.55, 0.4, 0.25]`).
2. Subplot 2 (`plt.subplot(1, 2, 2)`): Plot training accuracy (`[0.5, 0.7, 0.85, 0.95]`) and validation accuracy (`[0.45, 0.65, 0.8, 0.88]`).
3. Format titles, axis labels, legends, and grid lines for both subplots.